Report

Of

Java Full Stack With Project

Submitted

To

School of Computer Science and Engineering
IILM University, Greater Noida, U.P.

In partial fulfilment of the requirement of degree of

B.Tech CSE

By:

Roll Number of Student:2410030456

Name of Student: MOHD KHUBAIB

Batch:

2026-27

CANDIDATE'S DECLARATION

I, MOHD KHUBAIB, hereby declare that the internship report titled "Java Full Stack with Project" has been completed by me to fulfil the requirement for the award of the degree of Bachelor of Technology in Computer Science and Engineering. The work presented in this report is based on the internship project completed during the AICTE–EduSkills Virtual Internship Program. All references used in preparing this report have been appropriately acknowledged. I further declare that this report has not been submitted to any other institute for the award of any degree or diploma, and I shall be solely responsible for any copyright violation in this regard.

Signature
Student Name-Mohd Khubaib
Roll No-2410030456

ACKNOWLEDGEMENT

I express my sincere gratitude to AICTE and EduSkills for providing me with the opportunity to undertake this online virtual internship. The internship provided valuable exposure to Java Full Stack concepts and practical project development.

I am especially thankful to my industry mentor, Ms. Sagarabala Behera, for valuable guidance, continuous support, constructive feedback, and technical insights throughout the internship period. I also thank the School of Computer Science and Engineering, IILM University, Greater Noida, for its encouragement and academic support.

Finally, I express my gratitude to my parents and everyone who supported me during the successful completion of the internship and the preparation of this report.

Signature
Student Name-Mohd Khubaib
Roll No..2410030456

INTERNSHIP COMPLETION CERTIFICATE

TABLE OF CONTENTS

1. INTRODUCTION

Industry-oriented learning through internships plays an important role in bridging the gap between academic knowledge and practical software development. The AICTE–EduSkills Virtual Internship provided an opportunity to apply programming concepts in a structured project environment and to understand how technical requirements are converted into working software.

The internship was completed in the domain of Java Full Stack With Project over a period of eight weeks from June 2026 to July 2026. The project undertaken during the internship was “File Metadata Analyzer”. The project focuses on processing text input line-by-line and generating useful structural metadata such as line information, blank-line classification, word counts, character counts, and the longest line observed.

The project also emphasized dependency-free JSON generation through manual string construction using Java core APIs. This provided practical experience in file processing, string manipulation, data organization, and implementation under technical constraints.

2. ORGANIZATION PROFILE

EduSkills Academy is an EdTech and skill-development organization that provides virtual internship opportunities designed to expose students to industry-relevant technologies and project-based learning. The internship was delivered through an online/virtual mode using the EduSkills learning platform.

The internship certificate identifies the program as an 8-week internship in Java Full Stack With Project. The training covered HTML, CSS and Bootstrap, JavaScript and jQuery, Core Java and Advanced Java, Spring Framework, Spring Boot, Hibernate, MySQL, Git and version control, along with the File Metadata Analyzer project.

Organization Detail	**Information**
Organization	EduSkills Academy
Domain	EdTech & Skill Development
Mode	Online / Virtual
Duration	8 Weeks (June 2026 – July 2026)
Platform	EduSkills LMS

Role	Intern
Industry Mentor	Sagarabala Behera
Project	File Metadata Analyzer

3. INTERNSHIP OVERVIEW

The internship combined technical learning with project implementation. The learner worked on Java Full Stack topics and applied programming concepts to a focused text-analysis project. The project was developed as a modular processing engine in which each analytical function contributes to the final structured metadata output.

- Internship Mode: Online / Virtual
- Duration: 8 Weeks
- Domain: Java Full Stack With Project
- Role: Intern
- Industry Mentor: Sagarabala Behera, EduSkills Academy
- Project Title: File Metadata Analyzer
- Project Focus: Text-file structural metadata extraction and JSON generation.

4. PROBLEM STATEMENT

The File Metadata Analyzer is a text-processing engine designed to extract valuable structural metadata from text input. The engine operates line-by-line and analyzes the structure of the input through multiple modules. The required analysis includes sequential line enumeration, classification of blank and content lines, calculation of the number of words in each line, calculation of total characters in each line, and tracking of the longest line encountered during processing.

A key technical constraint of the project is the generation of JSON output without depending on external JSON libraries. The implementation therefore uses Java's List and Scanner APIs together with manual string construction. This constraint encourages a clear understanding of string handling, collection management, input processing, and output formatting.

5. PROJECT OBJECTIVES

1. To design and implement a solution for the specified File Metadata Analyzer problem.
2. To process text input sequentially on a line-by-line basis.
3. To classify lines as blank or content according to their whitespace content.
4. To calculate word counts for individual lines using whitespace-based word boundaries.
5. To calculate character counts for each line, including whitespace and special characters.
6. To track the maximum line length observed during processing.
7. To generate structured JSON output using manual string construction and Java core APIs.
8. To improve programming practice in modular problem solving, input handling, and dependency management.

6. SCOPE OF THE PROJECT

The present project is scoped to structural analysis of plain text input. It provides a foundation for automated file inspection by extracting simple but useful metadata from every processed line.

- Line-level indexing and sequential enumeration.
- Identification of blank lines and content lines.
- Whitespace-based word counting.
- Character-length calculation for each line.
- Continuous tracking of the longest line length.
- Generation of structured JSON output without external dependencies.

The current implementation is intentionally focused on plain text and the specified analytical modules. More advanced content understanding and support for additional file formats are outside the current scope but are identified as possible future enhancements.

7. TECHNOLOGIES AND TOOLS USED

Technology / Tool	**Use in Project**	**Purpose**
Java	Primary programming language	Implementation of processing

		modules
Java Collections Framework (List)	Result storage and ordering	Manage generated metadata elements
Java I/O / Scanner	Input processing	Read text input line-by-line
String Manipulation	Text analysis	Count words, characters, and build output
Manual JSON String Building	Output generation	Produce structured JSON without external libraries
Git & Version Control	Development practice	Version-control knowledge covered during internship
Spring / Spring Boot / Hibernate	Training domain	Java full-stack technologies covered during internship
HTML / CSS / Bootstrap / JavaScript / jQuery	Training domain	Web-development technologies covered during internship
MySQL	Training domain	Database technology covered during internship

8. SYSTEM ARCHITECTURE

The project follows a sequential processing architecture. Text input is read using Scanner, passed through a set of analytical modules, stored in a List where required, and finally assembled into a JSON-formatted output string.

Stage	**Component**	**Function**
1	Text Input	Receives text and reads it line-by-line.
2	Line Processing	Processes each line sequentially using Scanner.
3	Analysis Modules	Enumerates lines, detects blank lines, counts words and characters, and tracks maximum line length.
4	Result Collection	Stores generated result fragments in Java List where required.
5	JSON Output	Combines result fragments using manual string construction and prints structured output.

9. METHODOLOGY

The development of the File Metadata Analyzer was carried out systematically. First, the requirement to process text input line-by-line was established. Individual analytical components were then designed as distinct modules so that each component could perform one clearly defined function.

9.1 Line Enumeration

The Line Enumerator reads each input line and assigns a sequential index. The implementation maintains a counter and stores a structured result for each processed line. Empty lines can be skipped where the specific requirement calls for it.

9.2 Blank Line Classification

The Blank Line Classifier checks whether a line contains any non-whitespace characters. The use of trim().isEmpty() allows lines containing only spaces or tabs to be treated as blank, while lines containing actual content are classified as content.

9.3 Word Count

For each line, the implementation trims leading and trailing whitespace and splits the remaining text using one or more whitespace characters. Empty or whitespace-only lines receive a word count of zero.

9.4 Character Analysis

Character analysis uses the Java String length() operation. The count represents the total number of characters in the line, including whitespace and special symbols.

9.5 Longest Line Tracking

A running maximum is maintained while the input is processed. For every line, its length is compared with the maximum recorded so far, and the maximum is updated whenever a longer line is encountered.

9.6 JSON Generation

The final output is assembled manually rather than using libraries such as Jackson or Gson. Result fragments are represented as strings and combined with the required braces, brackets, quotation marks, colons, and commas to form the final JSON structure.

10. IMPLEMENTATION AND RESULTS

10.1 Line Enumerator

The Line Enumerator uses Scanner.hasNextLine() and Scanner.nextLine() to read input sequentially. A counter is incremented for each applicable line and a JSON fragment representing the line index is added to a List. The stored fragments are subsequently printed as a JSON array.

10.2 Blank Line Classifier

The classifier evaluates line.trim().isEmpty(). When the expression is true, the result is marked as blank; otherwise, it is marked as content. This approach handles empty lines and lines containing only whitespace.

10.3 Word Count

The word-count component first checks for an empty or whitespace-only line. For non-empty input, it uses split("\\s+") after trimming the line, so multiple spaces or tabs between words do not create incorrect empty word entries.

10.4 Character Count

The character-analysis module uses line.length() to determine the number of characters in each line. This directly satisfies the requirement that whitespace and special symbols are included in the count.

10.5 Longest Line Tracker

The tracker initializes the maximum length to zero and compares every new line's length against the current maximum. The output after each line therefore represents the largest line length observed up to that point in the analysis.

10.6 Result Summary

Module	Processing	Output / Result

Line Enumerator	Reads each line sequentially	Sequential line index
Blank Line Classifier	Checks whitespace content	blank / content classification
Word Counter	Uses whitespace boundaries	Word count per line
Character Analysis	Uses line length	Character count per line
Longest Line Tracker	Compares current length with maximum	Maximum length observed so far

11. LEARNING OUTCOMES

- Gained practical experience in line-by-line text processing using Java.
- Improved understanding of Java Scanner and List APIs for input processing and result management.
- Developed proficiency in manual string manipulation for structured JSON generation.
- Understood text-analysis algorithms involving counting, classification, and tracking.
- Learned to handle edge cases such as blank lines, multiple spaces, and whitespace-only input.
- Reinforced the importance of following technical constraints and controlling external dependencies.
- Gained exposure to the broader Java Full Stack technology stack covered during the internship.

12. CHALLENGES AND SOLUTIONS

12.1 Manual JSON Generation

Challenge: The project required JSON generation without relying on external libraries such as Jackson or Gson. Solution: JSON fragments were constructed carefully by concatenating the required structural characters and using Java List to manage the generated elements before final assembly.

12.2 Blank and Whitespace Lines

Challenge: Lines containing spaces or tabs had to be distinguished from lines containing meaningful content. Solution: String trimming and an emptiness check were used so that whitespace-only lines were classified correctly.

12.3 Word Boundaries

Challenge: Multiple spaces between words could lead to incorrect counting if processed directly. Solution: The line was trimmed and split using one or more whitespace characters, producing a reliable word count for the defined input model.

13. CONCLUSION

The File Metadata Analyzer successfully addresses the requirement of generating detailed structural metadata from text input through a modular, line-by-line processing engine. The project demonstrates how basic Java APIs can be combined to build a practical text-analysis solution.

The restriction against external JSON libraries strengthened the understanding of manual string construction, collection handling, and output formatting. Overall, the internship provided practical experience in applying programming concepts to a defined project problem and reinforced disciplined problem-solving and dependency-management practices.

14. FUTURE SCOPE

- Extend the analyzer to include more sophisticated content classification such as sentiment analysis or topic modeling.
- Add support for additional file formats beyond plain text.
- Enhance the output with statistical summaries such as average line length and distribution of line types.
- Provide richer metadata such as character-type distributions and additional text statistics.
- Integrate the analyzer into a web-based Java application for easier file upload and visualization of results.

15. BIBLIOGRAPHY / REFERENCES

Oracle Corporation. Java SE Documentation. Oracle Java Documentation.

Bloch, Joshua. Effective Java. Addison-Wesley Professional.

Goodrich, Michael T., Tamassia, Roberto, and Goldwasser, Michael H. Data Structures and Algorithms in Java. Wiley.

AICTE – EduSkills Virtual Internship Project Report: “File Metadata Analyzer”, supplied internship project material.

AICTE – EduSkills Virtual Internship Completion Certificate, issued 27 August 2026.

16. ANNEXURE

- Internship Completion Certificate – attached in this report.
- Course / Internship Project Report – File Metadata Analyzer.
- Other internship communication or offer documentation, if available.

The supplied completion certificate confirms successful completion of the 8-week Java Full Stack With Project internship and identifies the File Metadata Analyzer as the project completed during the program.